



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



TFG del Grado en Ingeniería
Informática

Gestión *offline* del banco
de preguntas de Moodle.

Documentación Técnica



Presentado por Miguel Alonso Alonso
en Universidad de Burgos - 7 de Junio
de 2026

Tutor: Dr. César Ignacio García Osorio



ÍNDICE DE CONTENIDOS

Apéndice A – Plan de Proyecto Software	4
A.1. Introducción	4
A.2. Planificación Temporal	4
A.2.1. Desarrollo Iterativo e Incremental	4
A.2.2. Especificación de los Sprints	4
A.3. Estudio de Viabilidad.....	6
A.3.1. Viabilidad Económica.....	6
A.3.2. Viabilidad Legal.....	9
Apéndice B – Especificación de Requisitos	11
B.1. Introducción	11
B.2. Objetivos Generales	11
B.3. Catálogo de Requisitos.....	11
B.4. Casos de Uso	13
Apéndice C – Especificación de Diseño	20
C.1. Introducción	20
C.2. Diseño de Datos	20
C.2.1. Jerarquía del Modelo de Preguntas	20
C.2.2. Estructura de Categorías	21
C.2.3. Archivos Multimedia Adjuntos.....	21
C.3. Diseño Procedimental	21
C.3.1. Flujo de Parseo XML	21
C.3.2. Flujo de Exportación a LaTeX.....	22
C.4. Diseño Arquitectónico	23
C.4.1. Organización en Paquetes	23
C.4.2. Arquitectura en Capas	23
C.4.3. Decisiones de Diseño Evaluadas y Descartadas	24
C.4.4. Aplicación de los Patrones de Diseño [3]	24
Apéndice D – Documentación Técnica de Programación	25
D.1. Introducción	25
D.2. Estructura de Directorios	25
D.3. Manual del Programador.....	25
D.3.1. Dependencias del Proyecto	26
D.3.2. Descripción de las Clases Principales.....	26



D.4. Compilación, Instalación y Ejecución del Proyecto	28
D.4.1. Prerrequisitos	28
D.5. Pruebas del Sistema	29
Apéndice E – Documentación de Usuario	30
E.1. Introducción.....	30
E.2. Requisitos del Usuario.....	30
E.3. Instalación.....	31
E.4. Manual del Usuario	31
E.4.1. Obtener el XML de un Banco de Preguntas desde Moodle	31
E.4.2. Descripción de la Interfaz Principal	31
E.4.3. Cargar un Banco de Preguntas XML.....	32
E.4.4. Navegar y Buscar	33
E.4.5. Editar el Banco de Preguntas.....	33
E.4.6. Visualización de Preguntas Cloze	34
E.4.7. Crear una Nueva Pregunta.....	34
E.4.8. Guardar los Cambios en XML	34
E.4.9. Exportar a LaTeX.....	34
E.4.10. Escenarios de Uso Avanzados.....	35
E.4.11. Preguntas Frecuentes y Resolución de Problemas	35
Apéndice F – Anexo de Sostenibilización Curricular.....	37
F.1. Introducción.....	37
F.2. Impacto del Proyecto en la Sostenibilidad.....	37
F.2.1. Dimensión Social.....	37
F.2.2. Dimensión Ambiental.....	38
F.2.3. Dimensión Económica.....	38
F.3. Competencias de Sostenibilidad Adquiridas y Aplicadas	38
Bibliografía	40

ÍNDICE DE FIGURAS

Figura A.2.2: Diagrama de Gantt sobre la elaboración del trabajo.....	6
Figura B.4: Diagrama de casos de uso	19
Figura C.3.1: Diagrama de secuencia sobre el parseo XML	22
Figura C.3.2: Diagrama de secuencia sobre la exportación a LaTeX.....	22
Figura E.4.2: Visualización de la interfaz principal	32



ÍNDICE DE TABLAS

Tabla A.3.1: Costes parciales y coste total del proyecto	8
Tabla A.3.2: Diferentes softwares con sus licencias y el uso dado.	9
Tabla B.4.1: CU-1 Cargar XML de Moodle.....	14
Tabla B.4.2: CU-2 Visualizar pregunta	14
Tabla B.4.3: CU-3 Editar banco de preguntas	15
Tabla B.4.4: CU-4 Crear nueva pregunta	15
Tabla B.4.5: CU-5 Exportar XML de Moodle.....	16
Tabla B.4.6: CU-6 Exportar a LaTeX.....	17
Tabla B.4.7: CU-7 Buscar y filtrar el banco	17
Tabla B.4.8: CU-8 Alternar modo de visualización Cloze	18
Tabla B.4.9: CU-9 Añadir categoría	18
Tabla C.2.1: Explicación de cada tipo de pregunta	21
Tabla C.4.1: Responsabilidad y composición de cada paquete del proyecto	23
Tabla D.2: Estructura de directorios del repositorio del proyecto	25
Tabla D.3.1: Dependencias del proyecto	26
Tabla D.3.2: Descripción de las clases principales del proyecto	28
Tabla D.4.1: Componentes a tener instalados.....	28
Tabla E.2: Componentes a tener descargados	30
Tabla E.4.11: Posibles problemas durante la manipulación de la aplicación con sus soluciones	36



APÉNDICE A – PLAN DE PROYECTO SOFTWARE

A.1. Introducción

Este apéndice recoge los aspectos de gestión y planificación que han guiado el desarrollo del presente TFG. Se presenta la planificación temporal, en donde se explica la metodología de desarrollo adoptada y la organización llevada a cabo en las diferentes iteraciones, junto al estudio de viabilidad económica y legal.

El proyecto se ha desarrollado a lo largo de aproximadamente seis meses, siguiendo un enfoque iterativo e incremental con revisiones periódicas con el tutor. El repositorio completo está disponible en:

<https://github.com/miguelito2003bcf/UBUTrabajoPreguntas>

A.2. Planificación Temporal

A.2.1. Desarrollo Iterativo e Incremental

El proyecto se gestionó siguiendo un enfoque de desarrollo iterativo e incremental inspirado en las metodologías ágiles, pero adaptado a las particularidades de un Trabajo Fin de Grado individual con orientación tutorial periódica. En lugar de planificar la totalidad de las funcionalidades en un documento de requisitos cerrado al inicio, el desarrollo se organizó en iteraciones de aproximadamente dos semanas de duración.

El trabajo comenzó a gestarse a principios de diciembre, estableciendo una etapa inicial denominada *Sprint 0* que se extendió hasta el inicio del segundo cuatrimestre. Este tiempo se dedicó a la investigación y documentación sobre aspectos técnicos nuevos del dominio del problema.

A partir de febrero, el desarrollo activo se estructuró en iteraciones cortas. Al final de cada *sprint* se realizaban reuniones de seguimiento para revisar conjuntamente todo lo elaborado durante esos días. De este modo, se disponía de un incremento funcional revisable que podía ser validado, permitiendo detectar tempranamente decisiones de diseño potencialmente problemáticas y reconducir el trabajo basándose en la retroalimentación directa por parte del tutor.

A.2.2. Especificación de los Sprints

La gestión temporal se estructuró en ciclos de aproximadamente 15 días, marcados por las reuniones de seguimiento periódico con el tutor. A continuación, se describe el contenido de cada *sprint* y el avance de la implementación.



- ***SPRINT 0 - INVESTIGACIÓN Y PLANTEAMIENTO (Diciembre 2025 – Febrero 2026)***

Fase inicial dedicada al estudio del dominio del problema. Se investigó la estructura secuencial del formato XML de exportación de Moodle. También se analizó el stack tecnológico a emplear, inclinándose por Java 17 y JavaFX para el desarrollo de la aplicación de escritorio. Por último, se plantearon las ideas fundamentales de la arquitectura del proyecto.

- ***SPRINT 1 - CONFIGURACIÓN INICIAL (Primera Quincena de Febrero de 2026)***

El objetivo de esta iteración fue la puesta en marcha del entorno de desarrollo y el repositorio remoto. Se redactó la información introductoria sobre el TFG en el *README* y se ejecutaron las primeras pruebas de control de versiones con Git, estableciendo el flujo de trabajo básico para los futuros *commits* [15].

- ***SPRINT 2 - ESTRUCTURA DEL REPOSITORIO (Segunda Quincena de Febrero de 2026)***

Se consolidó la estructura del entorno de desarrollo siguiendo las directrices dadas por el tutor. Se añadieron los archivos de ejemplo explicativos sobre la memoria y los anexos proporcionados en la asignatura de TFG a través de UBUVirtual y se realizó un ajuste arquitectónico de los directorios.

- ***SPRINT 3 - CORE LÓGICO Y UI BÁSICA (Primera Quincena de Marzo de 2026)***

Se subió la base del código fuente inicial con la primera parte del proyecto, que incluía el tratamiento de las categorías y de los distintos tipos de preguntas y el analizador sintáctico del archivo exportado desde Moodle.

- ***SPRINT 4 - INTERACTIVIDAD Y TESTING (Segunda Quincena de Marzo de 2026)***

Se introdujeron las primeras clases de tests para comprobar el correcto funcionamiento de la lógica. Se continuó puliendo la experiencia de usuario y la estabilidad del código. Se implementó el diálogo de confirmación ante el borrado de elementos en la interfaz para prevenir acciones accidentales.

- ***SPRINT 5 - ANÁLISIS ESTÁTICO Y BASE DE EXPORTACIÓN (Primera Quincena de Abril de 2026)***

Para asegurar la calidad del software, se integró el análisis de *SonarCloud* [12], añadiendo las *badges* de verificación correspondientes al *README* del proyecto. A nivel funcional, se introdujo una de las características clave del trabajo: la capacidad inicial de exportar los archivos en formato LaTeX.

- ***SPRINT 6 - DESARROLLO DE EXPORTACIÓN LATEX (Segunda Quincena de Abril de 2026)***

El desarrollo se centró en afianzar y extender el motor de exportación. Se añadieron nuevas funcionalidades específicas al sistema de exportación a LaTeX, adaptando el formato de salida a los requerimientos técnicos del banco de preguntas.



- **SPRINT 7 - REFINAMIENTO DEL FORMATO DE SALIDA (Primera Quincena de Mayo de 2026)**

Se consolidó la funcionalidad de exportación con la implementación de mejoras adicionales, permitiendo un manejo más robusto al generar los documentos exportados en formato LaTeX.

- **SPRINT 8 - INTERACTIVIDAD AVANZADA Y DOCUMENTACIÓN (Segunda Quincena de Mayo de 2026)**

Se implementaron funcionalidades avanzadas de visualización e interacción, como la funcionalidad para poder hacer clic en las opciones de las preguntas de tipo *multichoice*. En paralelo, se llevó a cabo una importante labor de documentación interna y mejora del código fuente.

- **SPRINT 9 - AJUSTES FINALES (Principios de Junio de 2026)**

Última iteración destinada a la entrega final. Se aplicaron los últimos retoques visuales en la interfaz gráfica que requerían atención. Se resolvieron las últimas modificaciones arquitectónicas pendientes y se actualizaron y ejecutaron por última vez las clases de tests para dar por validada y finalizada la implementación técnica del software.

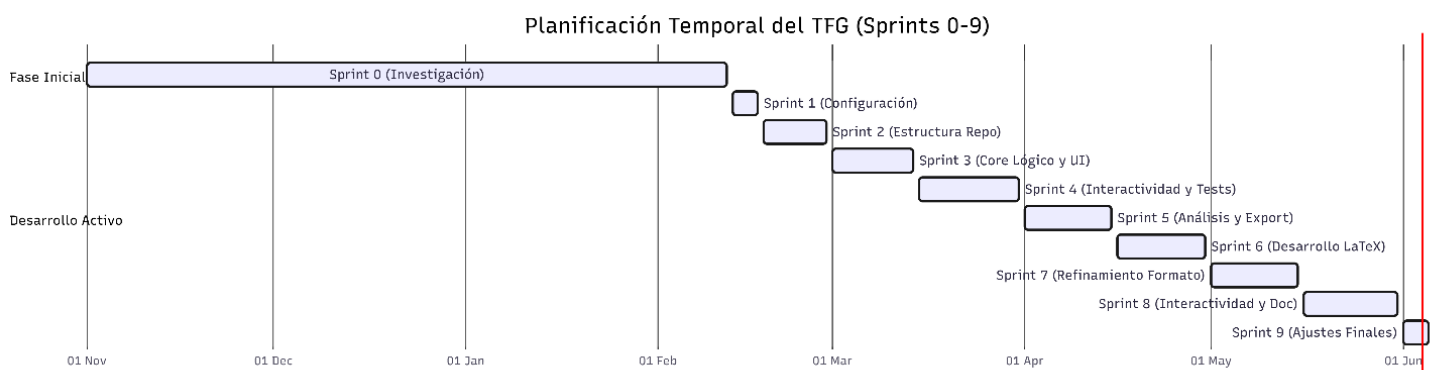


Figura A.2.2: Diagrama de Gantt sobre la elaboración del trabajo

A.3. Estudio de Viabilidad

En este apartado se hace un análisis de la viabilidad de mi proyecto desde distintos puntos de vista como son el económico y el legal, estudiando los distintos costes asociados al desarrollo y las distintas licencias de software utilizadas.

A.3.1. Viabilidad Económica

En este subapartado se calcularán los costes teóricos asociados a la elaboración del proyecto, desglosándose estos en costes personales, costes de hardware, costes de software y costes indirectos.



Costes de personal

Se estima una dedicación total de unas 600 horas distribuidas a lo largo de 6 meses, equivalente a una dedicación media de 25 horas semanales. Tomando como referencia el salario medio de un ingeniero junior en España que son unos 1930 € brutos/mes para 40 horas semanales, debemos de calcular el salario proporcional de la siguiente manera:

$$\frac{25 \text{ horas semanales}}{40 \text{ horas semanales}} * 1930 \text{ €/mes} = 1206 \text{ €/mes}$$

A este coste habría que añadirle las cotizaciones a la Seguridad Social a cargo de la empresa, el porcentaje actual ronda el 32%, por lo que el cálculo total para los 6 meses sería el siguiente:

$$1206 \text{ €/mes} * 1,32 * 6 \text{ meses} = 9551 \text{ €}$$

A esta cifra debe añadirse el coste estimado de la dedicación del tutor. Suponiendo una dedicación media de una hora semanal y un coste de 30 €/hora, el cálculo sería:

$$30 \text{ €/hora} * 4 \text{ horas/mes} * 1,32 * 6 \text{ meses} = 950 \text{ €}$$
$$\text{Coste total del personal} = 9551\text{€} + 950 \text{ €} = 10501 \text{ €}$$

Costes de hardware

Para la elaboración del proyecto se ha utilizado mi portátil personal valorado en unos 750 €, con una vida útil de 4 años. Por lo que el coste de amortización para los 6 meses del proyecto es:

$$\frac{750 \text{ €}}{4 \text{ años}} * \frac{6 \text{ meses}}{12 \text{ meses/año}} = 93,75 \text{ €}$$

No se requirió de ninguna infraestructura de servidor, ni servicios *cloud*, por lo que no tendremos que tener en cuenta ningún coste de hardware extra.



Costes de software

Todo el software empleado es de código abierto y gratuito. El único coste de software imputable corresponde a una licencia del sistema operativo Microsoft Windows 11 Home, con un precio aproximado de 145 €. Aplicando el mismo criterio de amortización utilizado para el hardware, esto también se amortiza en 4 años.

$$\frac{145 \text{ €}}{4 \text{ años}} * \frac{6 \text{ meses}}{12 \text{ meses/año}} = 18,15 \text{ €}$$

Costes indirectos

Estos costes incluyen los suministros necesarios para el desarrollo, suponiendo un consumo de electricidad medio de 50 kWh/mes a un precio aproximado de 0,17 €/kWh y un coste de conexión a internet de 30 €/mes, los costes indirectos para los 6 meses serán de:

$$\text{ELECTRICIDAD: } 50 \text{ kWh/mes} * 0,17 \text{ €/kWh} * 6 \text{ meses} = 51 \text{ €}$$

$$\text{INTERNET: } 30 \text{ €/mes} * 6 \text{ meses} = 180 \text{ €}$$

$$\text{TOTAL: } 51 \text{ €} + 180 \text{ €} = 231 \text{ €}$$

Coste total del proyecto

Hacemos la suma de todos los costes parciales para obtener una estimación del coste total del proyecto.

CONCEPTO	COSTE ESTIMADO (€)
Costes Personales	10501,00 €
Costes de Hardware	93,75 €
Costes de Software	18,15 €
Costes Indirectos	231,00 €
Coste Total	10843,90 €

Tabla A.3.1: Costes parciales y coste total del proyecto

Como podemos ver en la tabla, el coste total de la elaboración de nuestro proyecto sería de unos 10843,90 €.



A.3.2. Viabilidad Legal

El desarrollo de la aplicación se ha servido de diferentes bibliotecas, *frameworks* y herramientas de terceros. Para garantizar la viabilidad legal del proyecto, es fundamental analizar las licencias de estos componentes y asegurar que la licencia final escogida para el software sea compatible con las obligaciones de las tecnologías utilizadas.

En la *Tabla A.3.2* se presenta un resumen con las diferentes herramientas y softwares utilizados junto con el uso que se le ha dado y su licencia.

Software / Herramienta	Licencia	Tipo de uso
Java 17 (OpenJDK) [6]	GNU GPL v2 with Classpath Exception	Lenguaje y plataforma de ejecución
JavaFX 17 (OpenJFX) [5]	GNU GPL v2 with Classpath Exception	<i>Framework</i> de interfaz gráfica
Apache Maven 3	Apache License 2.0	Gestión de dependencias y construcción
Eclipse IDE [17]	Eclipse Public License 2.0	Entorno de desarrollo integrado
Git	GNU GPL v2	Control de versiones
Pandoc	GNU GPL v2+	Conversión HTML a LaTeX (proceso externo)
MathJax	Apache License 2.0	Renderizado de fórmulas matemáticas
JUnit Jupiter 5.10.0	Eclipse Public License 2.0	<i>Framework</i> de pruebas unitarias
MoodleWindowsInstaller	GNU GPL v3+	Entorno de pruebas local (desarrollo y evaluación)

Tabla A.3.2: Diferentes softwares con sus licencias y el uso dado.

Como se detalla en la tabla anterior, el ecosistema de desarrollo del proyecto se apoya íntegramente en herramientas y bibliotecas de código abierto. Para garantizar la correcta distribución del software resultante, se ha realizado una evaluación de compatibilidad prestando especial atención a las licencias de carácter más restrictivo:



- ***El núcleo de ejecución (Java y JavaFX):*** El aspecto jurídico más relevante del análisis recae en la *Classpath Exception* asociada a la licencia GNU GPL v2 de OpenJDK y OpenJFX. Esta cláusula es fundamental para el proyecto, ya que permite que nuestro código se enlace dinámicamente con estas bibliotecas sin verse obligado a heredar los términos de la GPL, siempre y cuando no se modifique el código fuente original de las mismas.
- ***Aislamiento de procesos externos (Pandoc):*** Aunque Pandoc está sujeto a la licencia GNU GPL v2+, su integración no presenta conflictos. El sistema invoca a Pandoc de forma aislada como un proceso externo (a través de la clase *ProcessBuilder*), una modalidad de ejecución que, a nivel legal, no constituye la creación de una obra derivada.
- ***Entornos de prueba locales (MoodleWindowsInstaller [16]):*** Herramientas con licencias como GNU GPL v3+ se han empleado exclusivamente como plataformas de evaluación y desarrollo en entorno local. Dado que no se han modificado ni se distribuyen junto con el código final del proyecto, sus restricciones no afectan a nuestra aplicación.
- ***Librerías permisivas:*** El resto de las dependencias, gestores y entornos (como MathJax, Maven, JUnit o Eclipse IDE) operan bajo licencias altamente permisivas (Apache License 2.0 y Eclipse Public License), diseñadas específicamente para integrarse sin fricciones legales en cualquier tipo de desarrollo.

Tras evaluar la naturaleza de las integraciones y validar que las herramientas externas y las bibliotecas base están correctamente aisladas o amparadas por excepciones, se concluye que no existen impedimentos legales para la distribución del software.

Por ello, se ha decidido publicar el código fuente de este proyecto bajo la Licencia MIT. Esta decisión técnica y legal maximiza la libertad de uso, modificación y distribución para futuros usuarios o investigadores, manteniendo una compatibilidad absoluta con el marco jurídico de todas las dependencias empleadas.



APÉNDICE B – ESPECIFICACIÓN DE REQUISITOS

B.1. Introducción

Este apéndice presenta la especificación formal de requisitos del sistema. Su objetivo es definir de manera precisa las capacidades que la aplicación debe ofrecer al usuario (requisitos funcionales) y los atributos de calidad bajo los que debe operar (requisitos no funcionales).

B.2. Objetivos Generales

Los objetivos generales que definen el propósito y alcance principal de la aplicación son los siguientes:

- Proporcionar al docente una herramienta de escritorio independiente de la plataforma Moodle para la gestión local de sus bancos de preguntas.
- Garantizar la fidelidad bidireccional con el formato XML nativo de Moodle [9], de modo que los bancos puedan importarse y exportarse sin pérdida de información.
- Ofrecer visualización enriquecida con soporte de HTML, imágenes Base64 y fórmulas matemáticas MathJax, replicando la experiencia visual de la plataforma.
- Facilitar la reestructuración interactiva del banco mediante *Drag & Drop* y operaciones de gestión masiva.
- Permitir la exportación a LaTeX en dos modalidades: examen en blanco para alumnos y solucionario para el docente.
- Garantizar la extensibilidad del sistema para nuevos tipos de pregunta o formatos de exportación sin modificar el código existente.

B.3. Catálogo de Requisitos

Requisitos Funcionales

Los requisitos funcionales (RF) especifican lo que el sistema debe hacer. Describen las funcionalidades, tareas y servicios que la aplicación final debe proporcionar al usuario para cumplir con su propósito.

- **RF-1: Importación XML:** El sistema debe cargar un fichero XML [13] proveniente de Moodle, extraer la jerarquía de categorías, instanciar preguntas de todos los tipos soportados y mostrar el árbol resultante. Se deben preservar los enunciados en formato HTML, las fracciones, la retroalimentación y los archivos adjuntos codificados en Base64.



- **RF-2: Exportación XML:** El sistema debe permitir exportar el árbol de categorías y preguntas a un fichero XML compatible con Moodle, preservando su estructura, los atributos específicos de cada tipo de pregunta y los archivos multimedia adjuntos.
- **RF-3: Visualización de preguntas:** La aplicación debe mostrar cada pregunta en un componente *WebView*, renderizando el contenido HTML con soporte para imágenes integradas en Base64 y fórmulas matemáticas mediante MathJax [4].
- **RF-4: Soporte para preguntas Cloze [8]:** El sistema debe soportar el procesamiento y visualización de preguntas tipo Cloze tanto en modo profesor, aplicando resaltado de sintaxis, como en modo alumno, generando *widgets* HTML interactivos.
- **RF-5: Gestión de categorías:** El sistema debe permitir al usuario crear, renombrar y eliminar categorías (ofreciendo la opción de reubicar su contenido), así como moverlas jerárquicamente utilizando una interfaz de arrastrar y soltar (*Drag & Drop*).
- **RF-6: Gestión de preguntas:** El usuario debe poder editar el nombre de las preguntas, eliminarlas mediante un cuadro de diálogo de confirmación y moverlas entre diferentes categorías a través de la funcionalidad de arrastrar y soltar (*Drag & Drop*).
- **RF-7: Creación de preguntas:** La aplicación debe proporcionar un cuadro de diálogo con un formulario dinámico para la creación de nuevas preguntas, adaptando automáticamente sus campos según el tipo de pregunta seleccionado por el usuario.
- **RF-8: Búsqueda y filtrado:** El sistema debe implementar un mecanismo que permita filtrar categorías y preguntas en la interfaz en tiempo real, utilizando tanto criterios de texto introducidos por el usuario como el tipo específico de pregunta.
- **RF-9: Exportación LaTeX:** El sistema debe permitir exportar el banco de preguntas a un fichero *.tex* que sea directamente compilable con la clase *exam* [11]. Este proceso debe incluir la extracción automática de imágenes y la conversión del contenido HTML a sintaxis LaTeX utilizando Pandoc [2].

Requisitos no Funcionales

Los requisitos no funcionales (RNF) especifican cómo debe comportarse el sistema, estableciendo restricciones y atributos de calidad. Describen las características globales que la aplicación final debe cumplir para garantizar su rendimiento, mantenibilidad y usabilidad.

- **RNF-1: Rendimiento:** El sistema debe ofrecer un tiempo de respuesta menor de 500 ms para la gestión de bancos de hasta 1.000 preguntas. Asimismo, el proceso de lectura de un fichero con 500 preguntas debe realizarse en menos de 5 segundos.



- **RNF-2: Portabilidad:** La aplicación debe poder ejecutarse de forma nativa y sin modificaciones en sistemas operativos Windows 10/11, macOS 11+ y Linux, requiriendo únicamente el entorno de ejecución de Java 17.
- **RNF-3: Usabilidad:** La interfaz gráfica debe respetar las convenciones estándar de las aplicaciones de escritorio. El diseño debe garantizar que las operaciones más frecuentes sean accesibles para el usuario en un máximo de dos interacciones.
- **RNF-4: Robustez:** El sistema debe ser capaz de gestionar la carga de archivos XML malformados o que contengan tipos de preguntas desconocidos, interceptando estos fallos para mostrar mensajes descriptivos al usuario sin que el programa termine de forma abrupta.
- **RNF-5: Mantenibilidad:** La arquitectura del código debe garantizar la separación de responsabilidades mediante gestores especializados y la aplicación de patrones de diseño documentados. Además, el proyecto debe mantener una calificación global de nivel A en el análisis estático continuo de *SonarCloud*.
- **RNF-6: Fidelidad XML:** El archivo XML generado por el motor de exportación debe ser estrictamente válido a nivel sintáctico y totalmente compatible a nivel semántico con Moodle.

B.4. Casos de Uso

Los casos de uso describen las interacciones entre un actor, en nuestro caso el docente, y el sistema para alcanzar un objetivo. A continuación, se presentan los casos de uso principales.

CU-1 Cargar XML de Moodle	
Versión	1.0
Autor	Miguel Alonso Alonso
Requisitos asociados	RF-1
Descripción	El docente abre un fichero XML de Moodle para visualizar y editar su banco de preguntas en la aplicación.
Precondición	La aplicación está en ejecución. El docente dispone de un fichero XML exportado desde Moodle.
Acciones	1. El docente hace clic en “Cargar XML de Moodle”. 2. El sistema muestra un FileChooser filtrado por .xml. 3. El docente selecciona el fichero. 4. El sistema invoca <code>XMLParser.parseMoodleXML(file)</code> y construye el árbol. 5. El sistema actualiza el <code>TreeView</code> con <code>TreeBuilder.createTreeItem()</code> , actualiza la etiqueta del fichero y habilita los botones de guardado y exportación.



Postcondición	El banco está cargado en memoria y visible. El árbol muestra la jerarquía completa con contadores actualizados en cada nodo.
Excepciones	E-1: Si el fichero es inaccesible o está malformado, el sistema muestra un <code>Alert.ERROR</code> con el mensaje de excepción y no modifica el estado previo.
Importancia	Alta

Tabla B.4.1: CU-1 Cargar XML de Moodle

CU-2 Visualizar pregunta	
Versión	1.0
Autor	Miguel Alonso Alonso
Requisitos asociados	RF-3, RF-4
Descripción	El docente selecciona una pregunta del banco para ver su contenido completo renderizado en el panel de detalle.
Precondición	Se ha completado CU-1. El docente ha seleccionado una categoría con preguntas.
Acciones	1. El docente hace clic sobre una pregunta en el <code>TableView</code> . 2. El sistema invoca <code>question.getDetails()</code> . 3. Se carga el HTML en el <code>WebView</code> con <code>loadContent()</code> . 4. El <code>WebView</code> renderiza el enunciado con imágenes y fórmulas. 5. Si la pregunta es <code>ClozeQuestion</code> , el <code>clozeToggle</code> se hace visible.
Postcondición	El panel de detalle muestra el contenido completo: enunciado renderizado, interfaz representativa del tipo y retroalimentación general si existe.
Excepciones	Ninguna: el sistema renderiza cualquier pregunta. Los enunciados con HTML malformado se muestran con el renderizado que WebKit sea capaz de generar.
Importancia	Alta

Tabla B.4.2: CU-2 Visualizar pregunta

CU-3 Editar banco de preguntas	
Versión	1.0
Autor	Miguel Alonso Alonso
Requisitos asociados	RF-5, RF-6
Descripción	El docente reorganiza el banco mediante operaciones de creación, renombrado, eliminación y reubicación de categorías y preguntas.
Precondición	Se ha completado CU-1.



Acciones	Flujo A - Mover preguntas: seleccionar preguntas y arrastrar sobre categoría destino. El sistema mueve las preguntas y refresca tabla y árbol. Flujo B - Crear categoría: clic en “Añadir Categoría” o menú contextual, el sistema crea la categoría y actualiza el árbol. Flujo C - Eliminar categoría: menú contextual “Eliminar Categoría” con confirmación requerida, preguntas y subcategorías se mueven al padre. Flujo D - Mover categoría <i>Drag & Drop</i> : arrastrar sobre otra categoría; diálogo con opciones “Como subcategoría” o “Fusionar preguntas”.
Postcondición	El árbol y la tabla reflejan la operación realizada. Los contadores de preguntas se actualizan.
Excepciones	E-3.1: Intentar eliminar la categoría raíz muestra <code>Alert.WARNING</code> sin realizar cambios. E-3.2: Drop sobre descendiente o sobre sí mismo se ignora.
Importancia	Alta

Tabla B.4.3: CU-3 Editar banco de preguntas

CU-4 Crear nueva pregunta	
Versión	1.0
Autor	Miguel Alonso Alonso
Requisitos asociados	RF-7
Descripción	El docente crea una nueva pregunta de cualquier tipo soportado dentro de una categoría del banco.
Precondición	Se ha completado CU-1. El docente tiene seleccionada una categoría destino.
Acciones	1. Clic en “Añadir Pregunta”. 2. El sistema muestra <code>AddQuestionDialog</code> con campos comunes. 3. El docente selecciona el tipo. 4. <code>buildSpecificSettingsPanel()</code> actualiza la sección específica dinámicamente. 5. El docente rellena los campos y confirma. 6. <code>QuestionFactory.createQuestion()</code> instancia el objeto, se añade a la categoría y se refresca la tabla.
Postcondición	La nueva pregunta aparece en la tabla. El árbol muestra el conteo actualizado. La pregunta es visible y exportable.
Excepciones	E-4: Si el docente cancela, no se realizan cambios.
Importancia	Media

Tabla B.4.4: CU-4 Crear nueva pregunta



CU-5 Exportar XML de Moodle

Versión	1.0
Autor	Miguel Alonso Alonso
Requisitos asociados	RF-2
Descripción	El docente guarda el estado actual del banco en un fichero XML compatible con Moodle.
Precondición	Se ha completado CU-1.
Acciones	1. Clic en “Guardar Cambios XML”. 2. FileChooser con filtro .xml. 3. El docente elige ruta y nombre. 4. <code>XMLExporter.exportMoodleXML()</code> serializa el árbol y escribe el fichero con UTF-8 e indentación. 5. <code>Alert.INFORMATION</code> de confirmación.
Postcondición	El fichero XML contiene el banco completo con todas las modificaciones y puede importarse en Moodle.
Excepciones	E-5: Error de escritura muestra <code>Alert.ERROR</code> con la traza de excepción.
Importancia	Alta

Tabla B.4.5: CU-5 Exportar XML de Moodle

CU-6 Exportar a LaTeX

Versión	1.0
Autor	Miguel Alonso Alonso
Requisitos asociados	RF-9
Descripción	El docente exporta el banco a un fichero .tex compilable con la clase exam, eligiendo entre solucionario o examen en blanco.
Precondición	Se ha completado CU-1. Pandoc está instalado en el <i>PATH</i> del sistema.
Acciones	1. Clic en “Guardar Cambios LaTeX”. 2. Diálogo de opciones: “Con respuestas (Solucionario)” o “Sin respuestas (Examen)”. 3. El docente elige la modalidad. 4. FileChooser con filtro .tex. 5. Se crea la carpeta <code>NOMBRE_imagenes</code> , se extraen imágenes Base64 a disco, se convierte HTML a LaTeX mediante Pandoc y se escribe el .tex. 6. <code>Alert.INFORMATION</code> de confirmación.
Postcondición	El .tex y la carpeta de imágenes permiten obtener un PDF compilando con overleaf <code>NOMBRE.tex</code> sin errores.
Excepciones	E-6.1: Si Pandoc no está disponible, se usa el HTML como texto alternativo y se emite <code>Alert.WARNING</code> . E-6.2: Error de escritura muestra <code>Alert.ERROR</code> .
Importancia	Alta



Tabla B.4.6: CU-6 Exportar a LaTeX

CU-7 Buscar y filtrar el banco	
Versión	1.0
Autor	Miguel Alonso Alonso
Requisitos asociados	RF-8
Descripción	El docente localiza categorías o preguntas concretas mediante las herramientas de búsqueda en tiempo real y el filtrado por tipo.
Precondición	Se ha completado CU-1 y hay al menos una categoría con preguntas.
Acciones	Flujo A - Buscar categoría: el docente escribe en “Buscar categoría...”. El árbol se filtra en tiempo real. Borrar restaura el árbol completo. Ctrl+clic colapsa/expande toda la rama. Flujo B - Buscar pregunta: escribir en “Buscar por nombre...”. La tabla se filtra en tiempo real. Flujo C - Filtrar por tipo: marcar tipos en el menú “Filtrar por Tipo”. El botón cambia a rojo con el recuento de filtros activos.
Postcondición	Las categorías o preguntas que no coinciden se ocultan temporalmente en la vista filtrada. El estado del banco en memoria no se modifica durante la búsqueda.
Excepciones	Ninguna: criterio sin coincidencias muestra vista vacía sin error.
Importancia	Media

Tabla B.4.7: CU-7 Buscar y filtrar el banco

CU-8 Alternar modo de visualización Cloze	
Versión	1.0
Autor	Miguel Alonso Alonso
Requisitos asociados	RF-4
Descripción	El docente alterna entre la vista de sintaxis Cloze y la vista previa interactiva del alumno para una pregunta Cloze seleccionada.
Precondición	Se ha completado CU-2 con una pregunta ClozeQuestion seleccionada.
Acciones	1. El CheckBox “Ver como el alumno (Renderizado)” está visible. 2. El docente activa el CheckBox. 3. <code>MODO_PREVIA_ALUMNO = true</code> y el WebView recarga con <code>widgets HTML</code> interactivos. 4. El docente desactiva el CheckBox. 5. <code>MODO_PREVIA_ALUMNO = false</code> y el WebView muestra los tokens resaltados.



Postcondición	El WebView muestra la representación correcta para cada modo. Los <i>widgets</i> en modo alumno son visualmente similares a los de Moodle.
Excepciones	Ninguna.
Importancia	Media

Tabla B.4.8: CU-8 Alternar modo de visualización Cloze

CU-9 Añadir categoría	
Versión	1.0
Autor	Miguel Alonso Alonso
Requisitos asociados	RF-5
Descripción	El docente crea una nueva categoría de primer nivel en el banco de preguntas.
Precondición	Se ha completado CU-1.
Acciones	1. Clic en “Añadir Categoría” sobre el árbol. 2. <code>TextInputDialog</code> solicita el nombre. 3. El docente introduce el nombre y confirma. 4. Se crea una nueva <code>Category</code> como subcategoría de la raíz. 5. El árbol se actualiza mostrando la nueva categoría en orden alfabético.
Postcondición	La nueva categoría aparece con contador (0 0) y puede seleccionarse para añadirle preguntas.
Excepciones	E-9: Cancelar o nombre vacío no realiza ningún cambio.
Importancia	Media

Tabla B.4.9: CU-9 Añadir categoría

A continuación, se presenta a través de la *Figura B.4* el diagrama de casos de uso:

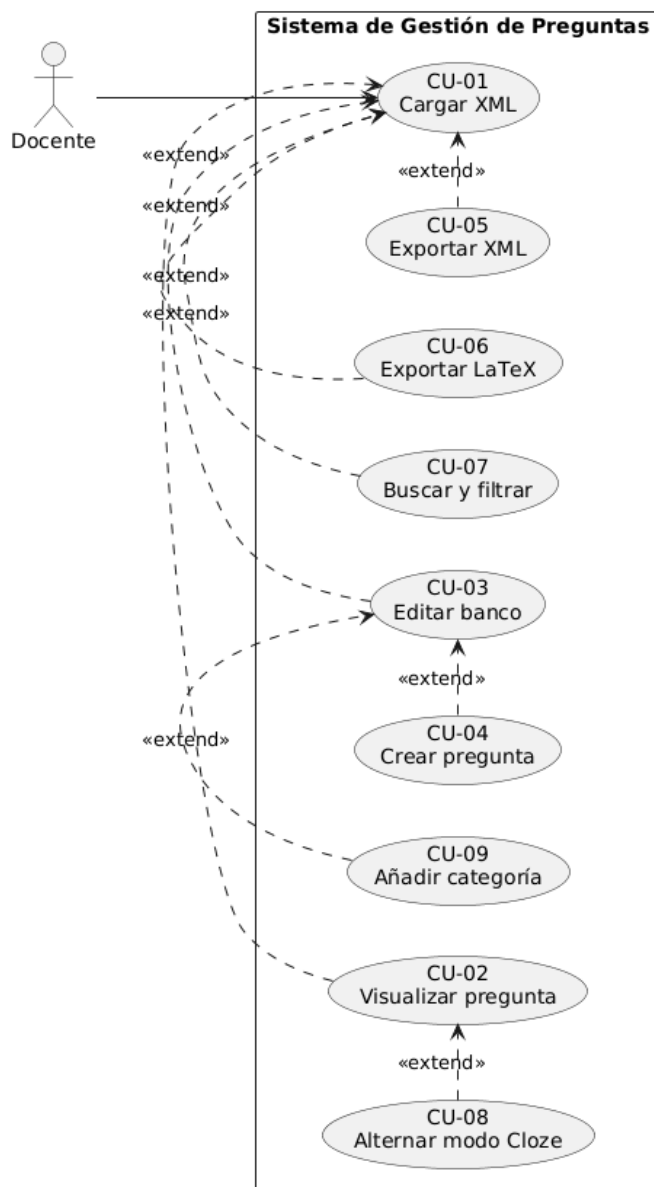


Figura B.4: Diagrama de casos de uso



APÉNDICE C – ESPECIFICACIÓN DE DISEÑO

C.1. Introducción

Este apéndice presenta el diseño técnico detallado del sistema. Se describen las decisiones arquitectónicas adoptadas, la estructura de datos del modelo de dominio, los flujos procedimentales de las operaciones clave y la organización arquitectónica de los paquetes. Se incluyen también las alternativas de diseño que se evaluaron y descartaron, justificando las decisiones tomadas.

C.2. Diseño de Datos

C.2.1. Jerarquía del Modelo de Preguntas

El núcleo del modelo de datos es la clase abstracta *Question*. Define los atributos compartidos: *type*, *name*, *text*, *defaultGrade*, *penalty*, *generalFeedback* y la lista de *MoodleFile* adjuntos. Proporciona los métodos concretos *getMoodleHeader()*, *getMoodleFooter()* y *processPluginFiles()*, que constituyen la infraestructura HTML compartida y declara los métodos abstractos *getDetails()* y *accept(QuestionVisitor)*.

En la siguiente *Tabla C.2.1* se explica cada clase propia del tipo de pregunta, junto a sus atributos específicos y una breve descripción [7].

Clase	Atributos específicos	Descripción del tipo
<i>MultipleChoiceQuestion</i>	<i>singleAnswer</i> : boolean, <i>shuffleAnswers</i> : boolean, <i>answers</i> : List<Answer>	Opción múltiple con una o varias respuestas correctas. Cada <i>Answer</i> tiene fracción, texto y retroalimentación.
<i>TrueFalseQuestion</i>	<i>trueAnswer</i> : Answer, <i>falseAnswer</i> : Answer	Verdadero/falso. Se serializa con valores literales <i>true/false</i> para compatibilidad con Moodle en cualquier idioma.
<i>ShortAnswerQuestion</i>	<i>caseSensitive</i> : boolean, <i>answers</i> : List<Answer>	Texto libre comparado con lista de respuestas aceptadas. <i>caseSensitive</i> controla la comparación.
<i>NumericalQuestion</i>	<i>answer</i> : Answer, <i>tolerance</i> : String	Valor numérico con margen de error. La tolerancia se preserva como cadena para evitar errores de redondeo.
<i>MatchingQuestion</i>	<i>pairs</i> : List<MatchingPair>	Emparejamiento de dos columnas. Cada <i>MatchingPair</i> contiene texto de enunciado y texto de respuesta correcta.



ClozeQuestion	MODO_PREVIA_ALUMNO: boolean (estático)	Pregunta anidada con subproblemas embebidos en el enunciado mediante la sintaxis Cloze de Moodle.
GenericQuestion	(sin atributos adicionales)	Tipo por defecto para ensayo y tipos no reconocidos por el parser.

Tabla C.2.1: Explicación de cada tipo de pregunta

C.2.2. Estructura de Categorías

La clase `Category` forma un árbol recursivo: cada nodo tiene `name`, `questions` y `subcategories`. La raíz es una `Category` con nombre “Banco de Preguntas” que actúa como nodo lógico de agrupación: `XMLOutputter` no la serializa como nodo `<question type="category">`, sino que la usa como punto de anclaje para comenzar el recorrido. Las categorías de primer nivel se serializan con ruta `$course$/top/NOMBRE` y las de niveles inferiores con rutas anidadas.

C.2.3. Archivos Multimedia Adjuntos

`MoodleFile` almacena: `name` (nombre del fichero), `path` (ruta virtual en Moodle), `encoding` (habitualmente `base64`) y `content` (contenido `Base64` como cadena). El método `processPluginFiles()` de `Question` sustituye las referencias `@@PLUGINFILE@@` del HTML por URIs `data:image/TYPE;base64,CONTENIDO`, con detección del tipo MIME por extensión (`png`, `jpg/jpeg`, `gif`, `svg`). Esto permite que las imágenes se muestren en el `WebView` sin acceso al servidor de Moodle.

C.3. Diseño Procedimental

Para mostrar como ocurre y se ve el flujo de la aplicación, a continuación, se presentan los diagramas de secuencia de dos de los procesos más relevantes como son el análisis sintáctico del archivo XML y la exportación a LaTeX.

C.3.1. Flujo de Parseo XML

La secuencia: `Main` llama a `FileManager.openXML(stage)`, `FileManager` muestra el `FileChooser`, obtiene el `File` y llama a `XMLParser.parseMoodleXML(file)`. `XMLParser` construye el árbol DOM completo, para cada nodo `category`, limpia la ruta y llama a `findOrCreateCategory()`. Para cada nodo pregunta, extrae atributos comunes y llama a `QuestionFactory.createQuestion()`, asigna `generalFeedback` y `files` y añade la pregunta a la categoría activa. `XMLParser` devuelve la categoría raíz, `FileManager` la encapsula en `Optional<Pair<Category,File>>` y `Main` actualiza el `TreeView`, la etiqueta del fichero y habilita los controles.

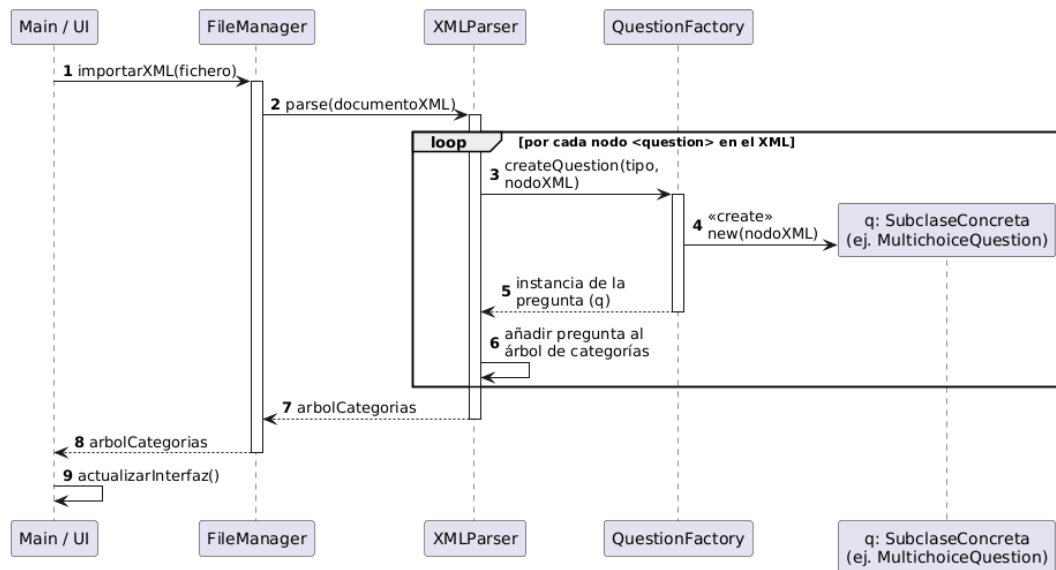


Figura C.3.1: Diagrama de secuencia sobre el parseo XML

C.3.2. Flujo de Exportación a LaTeX

Main llama a `FileManager.exportLaTeX(stage, currentRootCategory)`, `FileManager` muestra el diálogo de opciones y el `FileChooser`. `LaTeXExporter.exportToLaTeX()` escribe el preámbulo, crea el directorio de imágenes, instancia `LaTeXExportVisitor` y llama a `exportCategoryRecursive()`. Para cada pregunta `q`, llama a `q.accept(visitor)`. La subclase concreta invoca `visitor.visit(this)` (double dispatch). El visitor llama a `processTextForLatex()`, que aplica en orden: resuelve el `@@PLUGINFILE@@`, extracción de imágenes Base64 a disco, inserción de marcadores `XXSAANSSTARTXX/XXSAANSENDXX` para fragmentos Cloze, conversión HTML a LaTeX mediante Pandoc (ProcessBuilder con tres hilos) y `applyLatexFinalPatches()` para sanitizar el `output` y sustituir marcadores por comandos LaTeX definitivos.

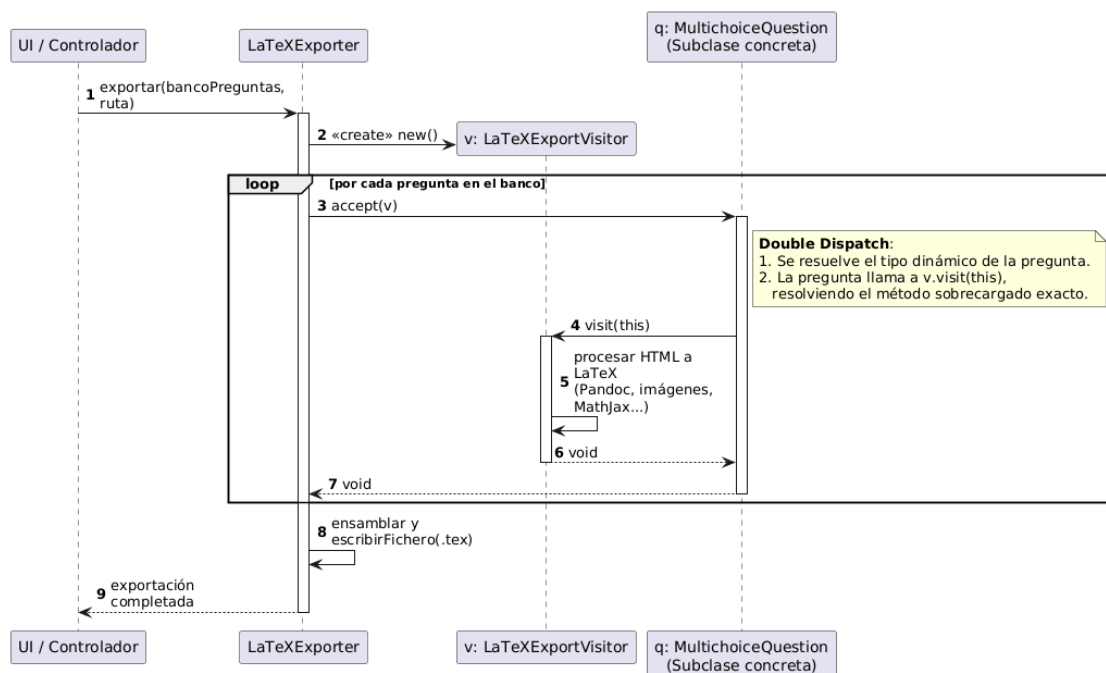


Figura C.3.2: Diagrama de secuencia sobre la exportación a LaTeX



C.4. Diseño Arquitectónico

C.4.1. Organización en Paquetes

En la siguiente *Tabla C.4.1* se muestra la composición de cada uno de los paquetes que forman el proyecto:

Paquete	Responsabilidad	Nombre de las Clases	Nº clases
moodleviewer	Capa de presentación JavaFX y coordinación global	Main, LayoutManager, TreeManager, TreeBuilder, TableManager, FileManager, AddQuestionDialog	7
moodleviewer.model	Modelo de dominio, independiente de JavaFX y formatos de serialización	Question, subclases, Category, Answer, MatchingPair, MoodleFile, QuestionVisitor	13
moodleviewer.parser	Transformación entre modelo en memoria y formatos externos	XMLParser, XMLExporter, QuestionFactory, LaTeXExporter, LaTeXExportVisitor	5
moodleviewer.util	Utilidades transversales compartidas por todas las capas	HtmlConstants	1

Tabla C.4.1: Responsabilidad y composición de cada paquete del proyecto

C.4.2. Arquitectura en Capas

El sistema sigue una arquitectura de tres capas con dependencias unidireccionales. La capa de presentación depende del modelo y del parser, la capa parser depende del modelo y el modelo no depende de ninguna otra capa. Este principio de inversión de dependencias garantiza que los cambios en la interfaz no obligan a modificar el modelo y que los cambios en el modelo no requieren tocar la interfaz. La interfaz `QuestionVisitor` actúa como punto de extensión que permite añadir operaciones al modelo sin modificar sus clases.



La clase Main actúa como registro de componentes, exponiendo los componentes JavaFX mediante getters públicos para que los gestores puedan acceder sin acoplamiento directo entre ellos. Los gestores (LayoutManager, TreeManager, TableManager, FileManager) son clases con métodos estáticos porque la aplicación es de ventana única con un único estado de sesión y no tiene sentido instanciarlos múltiples veces.

C.4.3. Decisiones de Diseño Evaluadas y Descartadas

- **SAX vs DOM:** SAX es más eficiente en memoria, pero la aplicación requiere acceso aleatorio continuo al árbol (el usuario puede seleccionar cualquier nodo, modificarlo y reexportarlo). DOM proporciona este acceso directo. El mayor consumo de memoria se asume a cambio de la flexibilidad necesaria para un editor interactivo.
- **Swing vs JavaFX:** Swing carece de un componente WebView con motor JavaScript y CSS modernos, lo que habría obligado a buscar bibliotecas de terceros para el renderizado de HTML con MathJax. JavaFX 17 incluye WebView basado en WebKit, que renderiza el HTML de los enunciados con JavaScript y CSS de forma nativa. La capacidad de renderizado web fue el factor determinante.
- **JDOM vs JAXP:** JDOM ofrece una API más amigable, pero añade una dependencia externa. Dado que el procesamiento XML del proyecto es directo (parseo secuencial y serialización de árbol), la API estándar JAXP [10] es suficiente. Este criterio sigue el principio de minimizar las dependencias de terceros cuando la funcionalidad requerida está disponible en la biblioteca estándar.

C.4.4. Aplicación de los Patrones de Diseño [3]

- **Factory Method (QuestionFactory):** createQuestion(type, name, text, grade, penalty, questionElement) recibe el tipo como cadena y devuelve la instancia polimórfica correcta mediante un switch. XMLParser solo conoce el tipo común Question. Añadir un nuevo tipo solo requiere añadir un case al switch y crear la subclase. XMLParser no se modifica, ilustrando el principio Abierto/Cerrado.
- **Visitor (QuestionVisitor / LaTeXExportVisitor):** QuestionVisitor declara visit() para cada subclase. LaTeXExportVisitor encapsula la serialización LaTeX en métodos visit() independientes por tipo. El double dispatch (q.accept(visitor) a visitor.visit(this)) resuelve la operación correcta en tiempo de ejecución sin casting ni instanceof. Añadir un formato de exportación alternativo solo requiere implementar un nuevo visitante, el modelo no se modifica.



APÉNDICE D – DOCUMENTACIÓN TÉCNICA DE PROGRAMACIÓN

D.1. Introducción

Este apéndice sirve como manual técnico para cualquier desarrollador que desee comprender, compilar, ejecutar o extender el proyecto. Se describe la estructura de directorios del repositorio, las dependencias del proyecto, la responsabilidad de cada clase, el proceso de compilación y ejecución con Maven y la suite de pruebas disponible.

El código fuente completo está disponible en:

<https://github.com/miguelito2003bcf/UBUTrabajoPreguntas>

D.2. Estructura de Directorios

A continuación, en la *Tabla D.2* se muestra y explica la estructura de directorios del repositorio principal del proyecto, que se puede encontrar en GitHub.

Carpeta/Archivo	Información
Carpeta doc	Carpeta en la que se encuentran los archivos de documentación del proyecto, como son la memoria y los anexos.
Carpeta src	Carpeta en la que se encuentran todos los archivos fuente con todos los contenidos programados y documentados en Java. En esta carpeta se encuentran también los tests.
Archivo pom.xml	Define las dependencias, <i>plugins</i> y configuración de construcción del proyecto Maven [14] que bibliotecas externas necesita el proyecto para funcionar a la perfección y como debe construirse desde el código fuente hasta el producto final.
Archivo .gitignore	Excluye del control de versiones los artefactos generados localmente, como bin/ y target/.

Tabla D.2: Estructura de directorios del repositorio del proyecto

D.3. Manual del Programador

Este apartado está creado para funcionar como una guía explicativa para un nuevo desarrollador que se vaya a poner a trabajar en nuestro proyecto.



D.3.1. Dependencias del Proyecto

A continuación, en la *Tabla D.3.1* se detallan las dependencias del proyecto que aparecen especificadas en el archivo pom.xml.

Artefacto Maven	Versión	Alcance	Propósito
org.openjfx:javafx-controls	17.0.6	compile	Componentes visuales JavaFX: TreeView, TableView, SplitPane, MenuButton, CheckBox, ComboBox, TextInputDialog, etc.
org.openjfx:javafx-web	17.0.6	compile	Módulo WebView con motor WebKit embebido para renderizado HTML con soporte de JavaScript y CSS.
org.junit.jupiter:junit-jupiter-api	5.10.0	test	API JUnit Jupiter 5 para escritura de tests unitarios.
org.junit.jupiter:junit-jupiter-engine	5.10.0	test	Motor de ejecución de JUnit Jupiter 5, necesario para Maven Surefire.

Tabla D.3.1: Dependencias del proyecto

D.3.2. Descripción de las Clases Principales

A continuación, en la *Tabla D.3.2* se describen aquellas clases consideradas más importantes dentro del proyecto.

Clase	Paquete	Responsabilidad principal
Main	moodleviewer	Punto de entrada JavaFX. Mantiene el estado global: currentRootCategory, activeTypeFilters y referencias a todos los componentes JavaFX. Expone componentes con getters públicos. Coordina gestores en start() y registra listeners en initBasicComponents().
LayoutManager	moodleviewer	Ensambla la escena: Toolbar superior con botones y etiqueta de fichero, SplitPane horizontal con el árbol de categorías y un SplitPane vertical anidado que separa tabla de preguntas y WebView.
TreeManager	moodleviewer	TreeManager configura el TreeView con celdas CategoryTreeCell que muestran nombre y contadores, implementan Drag & Drop de categorías con isValidDropTarget(), exponen menú contextual (añadir subcategoría, renombrar, eliminar) y gestionan Ctrl+clik para colapsar/expandir ramas.



TreeBuilder	moodleviewer	Fábrica estática de <code>TreeItem<Category></code> . <code>createTreeItem()</code> para árbol completo con subcategorías ordenadas alfabéticamente. <code>createFilteredTreeItem()</code> para árbol filtrado por texto.
TableManager	moodleviewer	Configura <code>TableView</code> : columnas nombre y tipo, selección múltiple, <i>Drag & Drop</i> de filas con <code>setOnDragDetected</code> , menú contextual “Editar Nombre” y “Eliminar Pregunta(s)” con confirmación.
FileManager	moodleviewer	Encapsula diálogos de apertura/guardado y llamadas a <code>XMLParser</code> , <code>XMLExporter</code> y <code>LaTeXExporter</code> . Devuelve <code>Optional<Pair<Category,File>></code> para distinguir operación completada de cancelada.
AddQuestionDialog	moodleviewer	Diálogo modal: <code>ComboBox</code> de tipo y <code>VBox</code> de sección específica reconstruida por <code>buildSpecificSettingsPanel()</code> . Clases internas: <code>MultichoiceOptionUI</code> , <code>MatchingPairUI</code> , <code>ShortAnswerOptionUI</code> , <code>NumericalOptionUI</code> .
Question	model	Clase abstracta raíz. Atributos comunes, <code>getMoodleHeader()</code> , <code>getMoodleFooter()</code> y <code>processPluginFiles()</code> . Contratos abstractos <code>getDetails()</code> y <code>accept(QuestionVisitor)</code> .
QuestionVisitor	model	Interfaz <code>Visitor</code> : 7 métodos <code>visit()</code> sobrecargados, uno por cada subclase concreta.
HtmlConstants	util	Clase final: 14 constantes CSS inline. <code>HTML_BODY</code> , <code>HTML_CARD</code> , <code>HEADER_FLEX</code> , <code>TITLE</code> , <code>BADGE</code> , <code>TEXT_BASE</code> , <code>LABEL_BOLD</code> , <code>FEEDBACK_GENERAL</code> , <code>FEEDBACK_WARNING</code> , <code>FLEX_ROW</code> , <code>FLEX_ROW_START</code> , <code>INPUT_BASE</code> , <code>TABLE_LAYOUT</code> , <code>CLOZE_CODE_BLOCK</code> , <code>CLOZE_RENDER_BLOCK</code> , <code>CLOZE_HIGHLIGHT</code> .
XMLParser	parser	Parsea XML de Moodle construyendo árbol de <code>Category</code> . <code>findOrCreateCategory()</code> navega/crea la jerarquía. <code>getNestedText()</code> y <code>getSimpleText()</code> encapsulan el acceso al DOM evitando capturas accidentales en nodos profundos.
QuestionFactory	parser	<i>Factory Method</i> estático. Switch sobre el tipo Moodle, extrae atributos específicos del DOM y devuelve la subclase <code>Question</code> correcta. Caso default devuelve <code>GenericQuestion</code> para tipos no reconocidos.
XMLExporter	parser	Serializa el árbol a DOM XML. <code>exportCategoryRecursive()</code> genera nodos <code><question type="category"></code> con rutas acumuladas. <code>writeQuestion()</code> serializa atributos comunes y específicos por tipo con <code>CDATA</code> para contenidos HTML. Transformer con UTF-8 e indentación de 2 espacios.
LaTeXExporter	parser	Escribe el preámbulo completo del <code>.tex</code> . Recorre el árbol llamando a <code>q.accept(visitor)</code> para cada pregunta. Ordena subcategorías alfabéticamente.



LaTeXExportVisitor	parser	Implementa QuestionVisitor. processTextForLatex(): resolución de @@PLUGINFILE@@, extracción imágenes Base64 a disco, marcadores Cloze, conversión HTML a LaTeX vía Pandoc, applyLatexFinalPatches().
--------------------	--------	--

Tabla D.3.2: Descripción de las clases principales del proyecto

D.4. Compilación, Instalación y Ejecución del Proyecto

En este apartado se indican los requisitos y los pasos a seguir para una correcta compilación, instalación y ejecución del proyecto.

D.4.1. Prerrequisitos

Asegurarse de tener instalados los siguientes componentes de la *Tabla D.4.1*:

Componente	Versión mínima	Enlace de descarga	Notas
JDK 17	17 LTS	https://adoptium.net/	JAVA_HOME debe apuntar al directorio de instalación. Verificar con java -version.
Apache Maven	3.8+	https://maven.apache.org/	mvn debe estar en el PATH. Verificar con mvn -version.
Pandoc	2.x+	https://pandoc.org/	Solo para exportación LaTeX. Verificar con pandoc --version.
Git	2.x	https://git-scm.com/	Para clonar el repositorio.

Tabla D.4.1: Componentes a tener instalados



Los pasos a seguir para una correcta compilación y ejecución del proyecto son los siguientes.

1. Clonar el repositorio:
`git clone https://github.com/miguelito2003bcf/UBUTrabajoPreguntas`
2. Nos movemos a la carpeta recién creada:
`cd UBUTrabajoPreguntas`
3. Compilar:
`mvn clean compile`
4. Ejecutar:
`mvn javafx:run`

La primera ejecución descarga las dependencias de JavaFX desde Maven Central. En ejecuciones posteriores las dependencias están en caché local y la aplicación arranca directamente.

D.5. Pruebas del Sistema

En el ámbito de las pruebas unitarias, se desarrollaron dos test utilizando el *framework* JUnit: el primero orientado a validar el correcto funcionamiento de la clase de categorías, y el segundo diseñado para verificar el comportamiento de los diferentes tipos de preguntas. Cabe destacar que no se expandió la cobertura de pruebas automatizadas con el fin de optimizar el tiempo disponible, priorizando la dedicación de recursos hacia otros aspectos críticos del desarrollo del proyecto.



APÉNDICE E – DOCUMENTACIÓN DE USUARIO

E.1. Introducción

Este apéndice sirve como manual de usuario de la aplicación “Editor Avanzado de Preguntas Moodle XML”. Guía al docente desde la instalación hasta el uso avanzado, describiendo paso a paso las operaciones disponibles, el significado de cada elemento de la interfaz, escenarios de uso representativos y la resolución de los problemas más frecuentes.

El perfil de usuario objetivo es el docente universitario con conocimientos básicos de informática y familiaridad con la plataforma Moodle. No se requieren conocimientos de programación ni del formato XML interno.

E.2. Requisitos del Usuario

Como ya mencionábamos en el apartado anterior es necesario tener en el dispositivo donde vayamos a hacer uso de la aplicación descargados los siguientes componentes:

Componente	Versión mínima	Enlace de descarga
Java 17 (JRE o JDK)	17 LTS	https://adoptium.net/
Apache Maven	3.8	https://maven.apache.org/
Pandoc	2.0+	https://pandoc.org/
Git	2.x	https://git-scm.com/

Tabla E.2: Componentes a tener descargados

Compatible con Windows 10/11, macOS 11+ y Linux. Requisitos recomendados: 4 GB RAM, procesador de doble núcleo. Para bancos de más de 1.000 preguntas, se recomienda 8 GB RAM.



E.3. Instalación

Los pasos a seguir para una correcta instalación y ejecución son los siguientes:

1. Clonar el repositorio:
`git clone https://github.com/miguelito2003bcf/UBUTrabajoPreguntas`
2. Nos movemos a la carpeta recién creada:
`cd UBUTrabajoPreguntas`
3. Compilar:
`mvn clean compile`
4. Ejecutar:
`mvn javafx:run`

Maven descarga las dependencias de JavaFX la primera vez. En ejecuciones posteriores, las dependencias están en caché local y la aplicación arranca directamente.

E.4. Manual del Usuario

E.4.1. Obtener el XML de un Banco de Preguntas desde Moodle

El punto de partida para usar la aplicación es disponer de un fichero XML exportado desde Moodle. Para obtenerlo desde una instalación de Moodle:

1. Acceder a Moodle con una cuenta de docente o administrador.
2. Navegar a la asignatura deseada e ir a banco de preguntas.
3. En el menú de administración del banco, seleccionar exportar.
4. Seleccionar el formato “Formato XML de Moodle”.
5. Marcar las categorías a exportar y hacer clic en “Exportar preguntas a fichero”.
6. Guardar el fichero .xml descargado en el sistema de ficheros local.

El fichero resultante contiene la jerarquía de categorías y todas las preguntas con sus atributos, imágenes adjuntas codificadas en Base64 y metadatos. Este es el fichero que se carga en la aplicación.

E.4.2. Descripción de la Interfaz Principal

- **Barra de herramientas (1):** botón “Cargar XML de Moodle”, etiqueta con el nombre del fichero activo, botón “Guardar Cambios XML” y botón “Guardar Cambios LaTeX”. Los botones de guardado y exportación se habilitan automáticamente al cargar un fichero.



- **Panel izquierdo - Árbol de categorías (2):** muestra la jerarquía del banco. Cada nodo muestra el nombre seguido de: número total de preguntas en la rama incluyendo subcategorías y número de preguntas directas de esa categoría. Sobre el árbol: campo “Buscar categoría...” y botón “Añadir Categoría”.
- **Panel superior derecho - Tabla de preguntas (3):** preguntas de la categoría seleccionada con columnas nombre y tipo, ordenables. Sobre la tabla: campo “Buscar por nombre...”, menú desplegable “Filtrar por Tipo” y botón “Añadir Pregunta”.
- **Panel inferior derecho - Vista de detalle (4):** contenido completo de la pregunta seleccionada renderizado como HTML. El control “Ver como el alumno (Renderizado)” es visible solo para preguntas Cloze.

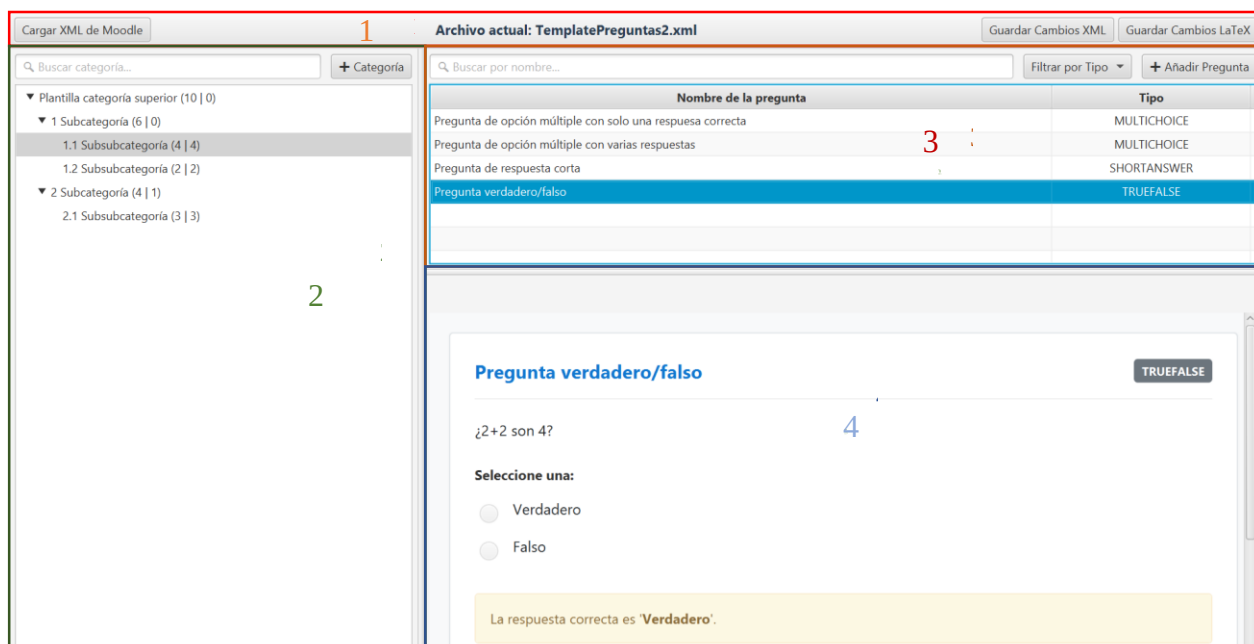


Figura E.4.2: Visualización de la interfaz principal

E.4.3. Cargar un Banco de Preguntas XML

1. Hacer clic en “Cargar XML de Moodle” en la barra de herramientas.
2. Seleccionar el fichero XML exportado desde Moodle en el diálogo (solo se muestran ficheros .xml).
3. La aplicación analiza sintácticamente el fichero y muestra el árbol. El nombre del fichero aparece en la barra y los botones de guardado se habilitan.
4. Hacer clic sobre una categoría para ver sus preguntas. Hacer clic sobre una pregunta para ver su contenido en el panel de detalle.



E.4.4. Navegar y Buscar

- **Búsqueda de categorías:** Escribir en “Buscar categoría...”. El árbol se filtra en tiempo real y solo las categorías cuyo nombre contiene el texto permanecen visibles. Borrar el campo restaura el árbol completo. Ctrl+clic sobre una categoría contrae o expande toda su rama.
- **Búsqueda de preguntas:** Escribir en “Buscar por nombre...”. La tabla se filtra en tiempo real. La búsqueda es insensible a mayúsculas/minúsculas y actúa sin necesidad de pulsar Enter.
- **Filtrado por tipo:** Hacer clic en “Filtrar por Tipo” y marcar uno o varios tipos. El botón cambia a rojo con el número de filtros activos entre paréntesis. Para eliminar todos los filtros, desmarcar todos los tipos.

E.4.5. Editar el Banco de Preguntas

- **Renombrar una categoría:**
 - ❖ Clic derecho sobre la categoría → “Renombrar” → Introducir el nuevo nombre → Confirmar.
- **Eliminar una categoría:**
 - ❖ Clic derecho → “Eliminar Categoría”. Diálogo de confirmación indicando que preguntas y subcategorías se trasladarán al padre. Confirmar con Aceptar.
- **Añadir subcategoría:**
 - ❖ Clic derecho sobre la categoría padre → “Añadir Subcategoría”, o clic en “Añadir Categoría” con la categoría padre seleccionada → Introducir nombre → Confirmar.
- **Mover preguntas:**
 - ❖ Seleccionar una o varias preguntas (Ctrl+clic para selección múltiple) → Arrastrar sobre la categoría destino en el árbol → Soltar. Las preguntas se trasladan automáticamente.
- **Mover una categoría:**
 - ❖ Arrastrar la categoría sobre el destino → Soltar. El diálogo ofrece “Como subcategoría” o “Fusionar preguntas”.



- **Renombrar o eliminar una pregunta:**

- ❖ Clic derecho sobre la pregunta → “Editar Nombre” o “Eliminar Pregunta(s)”. Las eliminaciones requieren confirmación.

E.4.6. Visualización de Preguntas Cloze

- **Modo profesor (CheckBox desactivado):** tokens Cloze resaltados con fondo gris y fuente monoespaciada azul. Facilita la revisión técnica de la sintaxis embebida.
- **Modo alumno (CheckBox activado):** cada token Cloze se sustituye por el *widget* HTML interactivo deshabilitado: campo de texto para SA y NM (con la respuesta correcta como *placeholder*), desplegable para MC horizontal, *radio buttons* para MC vertical, *checkboxes* para MR. Replica la vista que el alumno vería en Moodle.

E.4.7. Crear una Nueva Pregunta

1. Seleccionar la categoría destino en el árbol.
2. Clic en “Añadir Pregunta”.
3. Seleccionar el tipo en el desplegable. El formulario adapta la sección específica al tipo elegido.
4. Rellenar los campos comunes: nombre, enunciado HTML, calificación, retroalimentación y penalización.
5. Rellenar los campos específicos del tipo y confirmar con “Guardar cambios”.

E.4.8. Guardar los Cambios en XML

1. Clic en “Guardar Cambios XML”.
2. Elegir ruta y nombre del fichero (.xml).
3. Confirmación de éxito. El fichero puede importarse en Moodle mediante Banco de preguntas → Importar → Formato XML de Moodle.

E.4.9. Exportar a LaTeX

1. Clic en “Guardar Cambios LaTeX”.
2. Elegir modalidad: “Con respuestas (Solucionario)” o “Sin respuestas (Examen)”.
3. Elegir ruta y nombre del fichero .tex.
4. El sistema genera el .tex y la carpeta NOMBRE_imagenes con las imágenes adjuntas.
5. Para obtener el PDF, se podría realizar compilando el .tex con una instalación local de LaTeX o utilizando servicios como Overleaf, [18] tras subir el .tex como proyecto.

La carpeta NOMBRE_imagenes debe estar en el mismo directorio que el .tex al compilar.



E.4.10. Escenarios de Uso Avanzados

- **Escenario 1: Reorganización masiva de un banco heredado**

Un docente recibe un banco con todas las preguntas en una categoría sin estructura. Procedimiento: Cargar el XML. Crear categorías temáticas con el botón “Añadir Categoría”. Seleccionar en la tabla las preguntas de cada tema usando el buscador y Ctrl+clic para selección múltiple. Arrastrar cada grupo a su categoría. Guardar como nuevo XML. Esta operación, que en la interfaz web de Moodle requeriría decenas de operaciones individuales con recarga de página, se completa en la aplicación mediante unos pocos gestos de *Drag & Drop*.

- **Escenario 2: Generación de exámenes en dos versiones**

Un docente necesita el mismo examen con y sin soluciones para la misma convocatoria: Cargar el XML del banco. Exportar a LaTeX eligiendo “Con respuestas” para guardar como Examen_Solucionario.tex. Repetir eligiendo “Sin respuestas” para guardar como Examen_Alumnos.tex. Compilar ambos con Overleaf. Los dos documentos tienen exactamente la misma distribución y paginación. La única diferencia es que el solucionario muestra las respuestas correctas en un recuadro azul.

- **Escenario 3: Validación visual de preguntas Cloze complejas**

Un docente ha creado manualmente una pregunta Cloze con múltiples subproblemas. Para validar que la sintaxis es correcta: Cargar el XML. Seleccionar la pregunta. En modo profesor verificar que todos los tokens aparecen resaltados en la posición correcta. Activar el modo alumno para comprobar que los *widgets* HTML se generan correctamente. Si algún *token* no se resalta, indica un error de sintaxis que debe corregirse en Moodle antes de exportar.

E.4.11. Preguntas Frecuentes y Resolución de Problemas

A continuación, se presentan varios problemas que pueden aparecer con frecuencia durante la manipulación de nuestra aplicación con posibles soluciones.

Problema	Causa posible	Solución
La aplicación no arranca	Java 17 no instalado o JAVA_HOME no configurado.	Verificar con “java -version” (debe ser 17+). Instalar desde https://adoptium.net/
Error al cargar el XML	Fichero no es XML válido de Moodle o es de otro formato.	Verificar que se exportó desde Moodle con el formato “Formato XML de Moodle”.



Las fórmulas no se muestran	Sin conexión a internet, MathJax no carga desde CDN.	MathJax se carga desde <code>cdn.jsdelivr.net</code> . En entornos sin internet, descargar MathJax localmente y ajustar la URL en <code>Question.getMoodleHeader()</code> .
La exportación LaTeX falla	Pandoc no instalado o no está en el PATH.	Instalar Pandoc (pandoc.org). Verificar con <code>"pandoc -version"</code> . En Windows, reiniciar sesión para que el PATH se actualice.
Las imágenes no aparecen en el PDF	La carpeta <code>_imagenes</code> no está junto al <code>.tex</code> al compilar.	Asegurarse de que la carpeta <code>NOMBRE_imagenes</code> está en el mismo directorio que el <code>.tex</code> .
El PDF tiene errores de compilación LaTeX	El enunciado contiene caracteres especiales LaTeX sin escapar (<code>%</code> , <code>\$</code> , <code>#</code> , <code>&</code> , <code>_</code>).	Estos caracteres deben escaparse en LaTeX. Si el enunciado fue escrito para HTML puede contenerlos sin escapar. Corregir manualmente en el <code>.tex</code> generado.

Tabla E.4.11: Posibles problemas durante la manipulación de la aplicación con sus soluciones



APÉNDICE F – ANEXO DE SOSTENIBILIZACIÓN CURRICULAR

F.1. Introducción

Siguiendo las “Directrices para la introducción de la Sostenibilidad en el Curriculum” de la CRUE [19], este apéndice presenta una reflexión personal sobre cómo el presente Trabajo de Fin de Grado se alinea con los principios del desarrollo sostenible en sus dimensiones social, ambiental y económica y sobre las competencias de sostenibilidad adquiridas y aplicadas durante su desarrollo.

A primera vista, el desarrollo de una herramienta de escritorio para la gestión de bancos de preguntas puede parecer alejado de los debates habituales sobre sostenibilidad. Un análisis más profundo revela conexiones relevantes con los tres pilares del desarrollo sostenible, así como con la responsabilidad ética del ingeniero de software ante las implicaciones sociales y ambientales de su trabajo.

F.2. Impacto del Proyecto en la Sostenibilidad

F.2.1. Dimensión Social

El impacto social más directo radica en la mejora de la calidad de la educación universitaria. La gestión de los bancos de preguntas es una tarea recurrente y costosa en tiempo para el profesorado. Una herramienta que reduce ese coste operativo libera tiempo que el docente puede invertir en el diseño pedagógico de las evaluaciones y en la retroalimentación a los estudiantes. Desde la perspectiva de la equidad educativa, la herramienta contribuye a democratizar el acceso a evaluaciones de calidad: al facilitar la reutilización de bancos de preguntas estructurados, se reduce la dependencia de recursos externos y se empodera al docente para construir evaluaciones adaptadas al contexto de sus estudiantes.

El carácter de código abierto del proyecto tiene una dimensión social relevante: cualquier institución, con independencia de sus recursos económicos, puede adoptar, adaptar y mejorar la herramienta. La publicación del código fuente con documentación técnica detallada facilita que otros desarrolladores o docentes con conocimientos de programación puedan extenderla para necesidades específicas.

Desde el punto de vista de la privacidad, el diseño de la aplicación como herramienta offline que opera exclusivamente sobre ficheros locales garantiza que los contenidos del banco permanecen bajo control exclusivo del docente, sin ningún dato enviado a servidores externos.



F.2.2. Dimensión Ambiental

La naturaleza de escritorio y la operación offline tienen una ventaja ambiental directa frente a las alternativas web. Una sesión de trabajo reorganizando un banco de preguntas a través de la interfaz web de Moodle implica decenas de peticiones HTTP a un servidor remoto, procesamiento en servidor, transmisión de datos por red y renderizado en el cliente. La misma sesión realizada con esta aplicación implica únicamente procesamiento local, sin tráfico de red. Para un docente que realice esta tarea regularmente, el ahorro energético acumulado puede ser significativo.

En el proceso de desarrollo, la elección de herramientas locales minimizó el consumo energético frente a entornos de desarrollo en la nube.

La decisión de generar documentos .tex compilables localmente en lugar de recurrir a servicios *cloud* para la generación de documentos PDF delega el procesamiento en la máquina del docente en el momento exacto en que se necesita, sin añadir carga a infraestructura remota compartida.

F.2.3. Dimensión Económica

En el plano individual, la herramienta reduce el tiempo dedicado a tareas de gestión técnica del banco de preguntas. En el plano institucional, puede reducir la carga sobre los servidores de Moodle durante las operaciones masivas de reorganización, que en la interfaz web generan picos de carga en el servidor de base de datos.

La construcción exclusiva sobre tecnologías de código abierto elimina cualquier coste de licencia de software para la institución que adopte la herramienta. El único coste de adopción es el tiempo de configuración del entorno Java, que en la mayoría de los entornos universitarios ya está disponible. Esta característica contrasta con herramientas propietarias de gestión de evaluaciones que pueden suponer costes de licencia de varios miles de euros anuales.

F.3. Competencias de Sostenibilidad Adquiridas y Aplicadas

- **SOS1 - Contextualización crítica del conocimiento:**

El proyecto partió de un análisis del contexto real del problema: las limitaciones de la interfaz web de Moodle para la gestión de bancos de gran volumen y sus implicaciones para la productividad docente. Esta contextualización informó activamente las decisiones de diseño más importantes: la elección de una herramienta de escritorio offline, el soporte de todos los tipos de pregunta, la bidireccionalidad con el XML original y la integración de Pandoc para documentos físicos de calidad tipográfica. La creación de una instancia local de Moodle como entorno de pruebas fue, en sí misma, una expresión de esta competencia: comprender el problema requería trabajar con el sistema real, no con suposiciones sobre su comportamiento.



- ***SOS2 - Utilización sostenible de recursos:***

A lo largo del desarrollo se priorizó la reutilización sobre la reinención. La integración de Pandoc para la conversión HTML A LaTeX es el ejemplo más claro: una herramienta madura y robusta de código abierto sustituye a un parser que habría consumido semanas de desarrollo. La misma filosofía se aplicó con MathJax para el renderizado de fórmulas y con JAXP DOM de la biblioteca estándar del JDK. El principio DRY aplicado al propio código es también una forma concreta de gestión sostenible de recursos de desarrollo.

- ***SOS3 - Participación en procesos comunitarios:***

La publicación del proyecto en GitHub, con análisis de calidad públicamente visible en *SonarCloud* y documentación técnica detallada, son manifestaciones de una cultura de desarrollo transparente orientada a la comunidad. El código permite que otros estudien la aplicación de patrones de diseño en un contexto real, la adapten a sus necesidades institucionales y contribuyan con mejoras. En un nivel más amplio, este TFG es una contribución al ecosistema de herramientas de código abierto que complementan Moodle, la plataforma LMS de código abierto más utilizada del mundo con más de 400 millones de usuarios [1].

- ***SOS4 - Aplicación de principios éticos:***

El respeto por la privacidad de los datos educativos estuvo presente como principio de diseño desde el inicio. La elección de que la aplicación opere offline sin conectarse a servidores externos, sin telemetría y sin requerir autenticación en servicios de terceros, es una decisión ética deliberada que protege la confidencialidad de los contenidos del banco de preguntas. La elección de la licencia MIT refleja un compromiso ético con la accesibilidad: permite a cualquier institución, incluyendo aquellas con restricciones legales sobre el software que pueden distribuir, adoptar la herramienta sin barreras.



BIBLIOGRAFÍA

- [1] LMS Daily. *Moodle LMS now officially powers 400 million users around the world*. LMS Daily, noviembre 2023. Disponible en: <https://lmsdaily.com/moodle-lms-now-officially-powers-400-million-users-around-the-world/> [Internet; Accedido: 31-05-2026]
- [2] J. MacFarlane. *Pandoc: a universal document converter*. Universidad de California Berkeley, 2006–2025. Disponible en: <https://pandoc.org/> [Internet; Accedido: 31-05-2026]
- [3] E. Gamma, R. Helm, R. Johnson y J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994. ISBN: 0-201-63361-2.
- [4] MathJax Consortium. *MathJax: Beautiful and accessible math in all browsers*. Disponible en: <https://www.mathjax.org/> [Internet; Accedido: 31-05-2026]
- [5] Oracle Corporation / OpenJFX. *JavaFX: Getting Started with JavaFX (Release 8 / JDK 17)*. Oracle Documentation, 2024. Disponible en: <https://docs.oracle.com/javase/8/javafx/get-started-tutorial/jfx-overview.htm> [Internet; Accedido: 31-05-2026]
- [6] Oracle Corporation. *Java SE 17 API Specification*. Disponible en: <https://docs.oracle.com/en/java/javase/17/docs/api/> [Internet; Accedido: 31-05-2026]
- [7] Moodle HQ. *Moodle Documentation: Question types*. Moodle Docs, 2024. Disponible en: https://docs.moodle.org/en/Question_types [Internet; Accedido: 31-05-2026]
- [8] Moodle HQ. *Moodle Documentation: Embedded answers (Cloze) question type*. Moodle Docs, 2024.
Disponible en: [https://docs.moodle.org/en/Embedded_Answers_\(Cloze\)_question_type](https://docs.moodle.org/en/Embedded_Answers_(Cloze)_question_type) [Internet; Accedido: 31-05-2026]
- [9] Moodle HQ. *Moodle Documentation: Moodle XML format*. Moodle Docs, 2024. Disponible en: https://docs.moodle.org/en/Moodle_XML_format [Internet; Accedido: 31-05-2026]
- [10] Oracle Corporation. *Java API for XML Processing (JAXP)*. Disponible en: <https://docs.oracle.com/javase/8/docs/technotes/guides/xml/jaxp/> [Internet; Accedido: 31-05-2026]
- [11] Paquet, L. et al. *exam — A class for typesetting examinations*. CTAN. Disponible en: <https://ctan.org/pkg/exam> [Internet; Accedido: 31-05-2026]
- [12] SonarSource. *SonarQube Cloud — Online Code Review as a Service Tool*. Disponible en: <https://www.sonarsource.com/products/sonarqube/cloud/> [Internet; Accedido: 31-05-2026]
- [13] W3C. *Extensible Markup Language (XML) 1.0 (Fifth Edition)*. W3C Recommendation, noviembre 2008. Disponible en: <https://www.w3.org/TR/xml/> [Internet; Accedido: 31-05-2026]
- [14] Apache Software Foundation. *Apache Maven Project*. Disponible en: <https://maven.apache.org/> [Internet; Accedido: 31-05-2026]
- [15] Conventional Commits. *Conventional Commits — A specification for adding human and machine readable meaning to commit messages*. Disponible en: <https://www.conventionalcommits.org/en/v1.0.0/> [Internet; Accedido: 31-05-2026]
- [16] Moodle HQ. *MoodleWindowsInstaller — Moodle for Windows*. Disponible en: <https://download.moodle.org/windows/> [Internet; Accedido: 31-05-2026]



[17] Eclipse Foundation. *Eclipse IDE for Java Developers*. Disponible en: <https://www.eclipse.org/ide/> [Internet; Accedido: 31-05-2026]

[18] Overleaf. Overleaf — Online LaTeX editor. Disponible en: <https://www.overleaf.com/> [Internet; Accedido: 31-05-2026]

[19] CRUE Universidades Españolas. Directrices para la introducción de la Sostenibilidad en el Curriculum. Disponible en:

https://www.crue.org/wp-content/uploads/2020/02/Directrices_Sostenibilidad_Crue2012.pdf
[Internet; Accedido: 31-05-2026]